Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Positionnement du module

Dans la formation FISA à l'EIGSI

- 3A Algorithmique et Programmation, Prog. Orientée Objet
- 4A Modélisation Orientée Objet, Bases de Données
- Options

En tant qu'ingénieur.e généraliste

- Aujourd'hui, à l'ère du numérique
  - l'aspect programmation va être/est quasi-déjà une connaissance commune
- Une grande partie de votre travail s'effectuera via le Système d'Information (SI) de votre entreprise
  - pour notamment développer/adapter vos outils de travail
  - communiquer avec des développeurs sur vos besoins ou ceux du client

Équipe pédagogique

Référant de module :

- Corwin FEVRE : `corwin.fevre@eigsi.fr`
- Bureau : 1er étage, couloir de gauche (face au forum), deuxième porte à droite (en face des admissions)

$\Rightarrow$ N'hésitez pas à passer pour toute question ou idée

Intervenants :

- Thomas RAIMBAULT,
- Alix NGARI LENDOYE

Format du module

Face à face

- 5 CM (7,5h)
- 20 TD (30h) avec son ordinateur (personnel ou pro)

TD

- séances doubles (2 x 1h30 groupées)
- TD1-2, TD3-4, ..., TD19-20

Travail personnel

- $\approx$ 30h en individuel ou en groupe
- finir et refaire les exercices de TD (évidemment !)
- + tests en « auto-entrainement » disponibles

Déroulé

Les bases fondamentales (variables, tests cond., boucles, fonctions)	
CM1	TD1-2
Tableaux et Listes	
CM2	TD3-4
Récursivité, Complexité	
CM3	TD5-6, TD7-8
Algorithmique, Structures de données	
CM4 - 5	TD9-10, TD11-12, TD13-14, TD15-16
Problème noté	
—	TD17-18, TD19-20 + travail personnel

Langage de programmation utilisé : **Python**

Évaluation

50% Contrôle continu (CC)

- 10% Contrôle en TD
- 40% Problème

50% Examen

- sur papier en salle de DS
- questions de cours
- questions de code
- résolution d'un problème avec une approche algorithmique

En TD

- Travail sur son ordinateur
  - pas d'installation nécessaire (via navigateur - Google Colab)
  - ne pas oublier son chargeur

- Sujets de TD et corrections disponibles sur Moodle (au fur et à mesure)

- Contrôles en fin de TD
  - objectif : vérifier l'acquisition de **réflexes** de programmation sans IA
    $\Rightarrow$ il faut s'entraîner ! !
  - format : code à écrire sur papier, sur feuille libre (vérification syntaxe)
  - en début de séance double, environ entre le TD3 et TD16

ÉCOLE D'INGÉNIEURS
LA ROCHELLE • CASABLANCA

Algorithmique et programmation

Cours 1 : Introduction à l'algorithmique

FISA 3 – S5 – 2025
Référant : FEVRE Corwin – fevre@eigsi.fr
Intervenants : Thomas RAIMBAULT & Alix NGARI LENDOYE

Qu’est ce qu’un algorithme ?

Un algorithme amène une solution à un problème

Une notion très répandue dans la vie courante, par exemple :

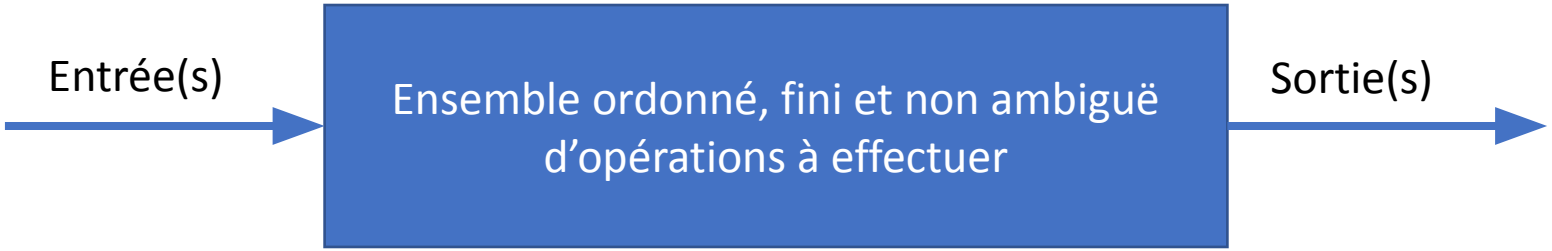
- d'une recette de cuisine,
- d'un mode d'emploi,
- d'une notice de montage,
- d'une partition musicale,
- d'un texte de loi,
- d'un itinéraire routier.





Qu’est ce qu’un algorithme ?

Un algorithme est une suite d'opérations, qu'on nomme **instructions**, à effectuer pour atteindre un objectif (résoudre un problème, cuisiner un plat, faire se déplacer un robot, etc.). On dit que ces instructions doivent être **exécutées**.



La **construction d’un algorithme** implique de se poser des questions sur son **efficacité** :

- A t-il toujours une **fin** ?
- Est-il **correct**, à t’il été testé ?
- Est-il **déterministe** ? (même entrée ⇒ même sortie)
- Est-il **rapide** ? Peut-il être plus rapide ? Dans quelles conditions ? (Complexité en **temps**)
- Taille **mémoire** utilisée ? Peut-il en utiliser moins ? (Complexité en **espace**)

Rapide historique

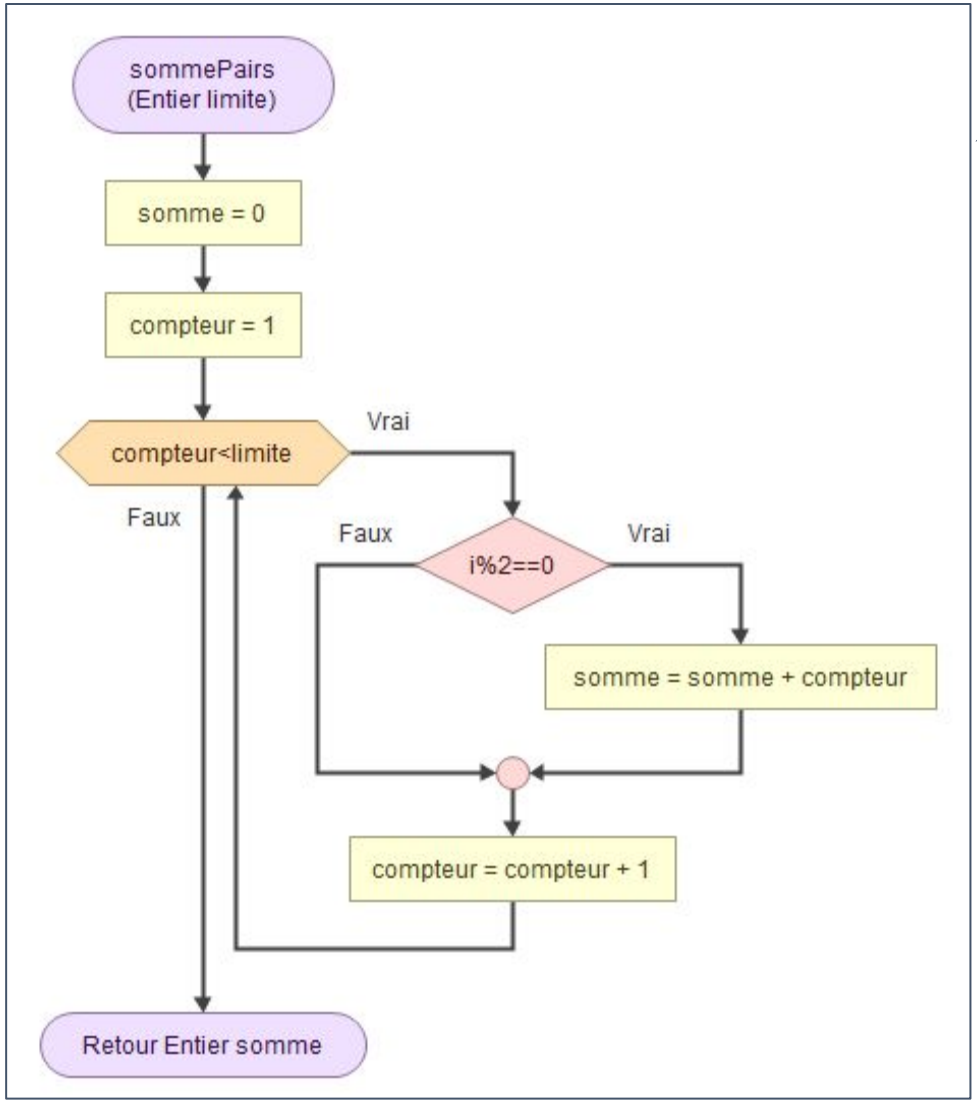


- 8ème siècle, un mathématicien nommé **Al-Khwarizmi**, a donné son nom à la notion d'algorithme.

- Son besoin était de **traduire et expliquer** un livre de mathématiques venu d'Inde pour que les résultats et les méthodes exposés dans ce livre soient **compréhensibles et applicables** par tout autre mathématicien, **sans ambiguïté.**

Notion de langage : Pour être compris, un algorithme doit être écrit dans un langage commun.

Différentes représentations d'un algorithme



Diagramme

Programme

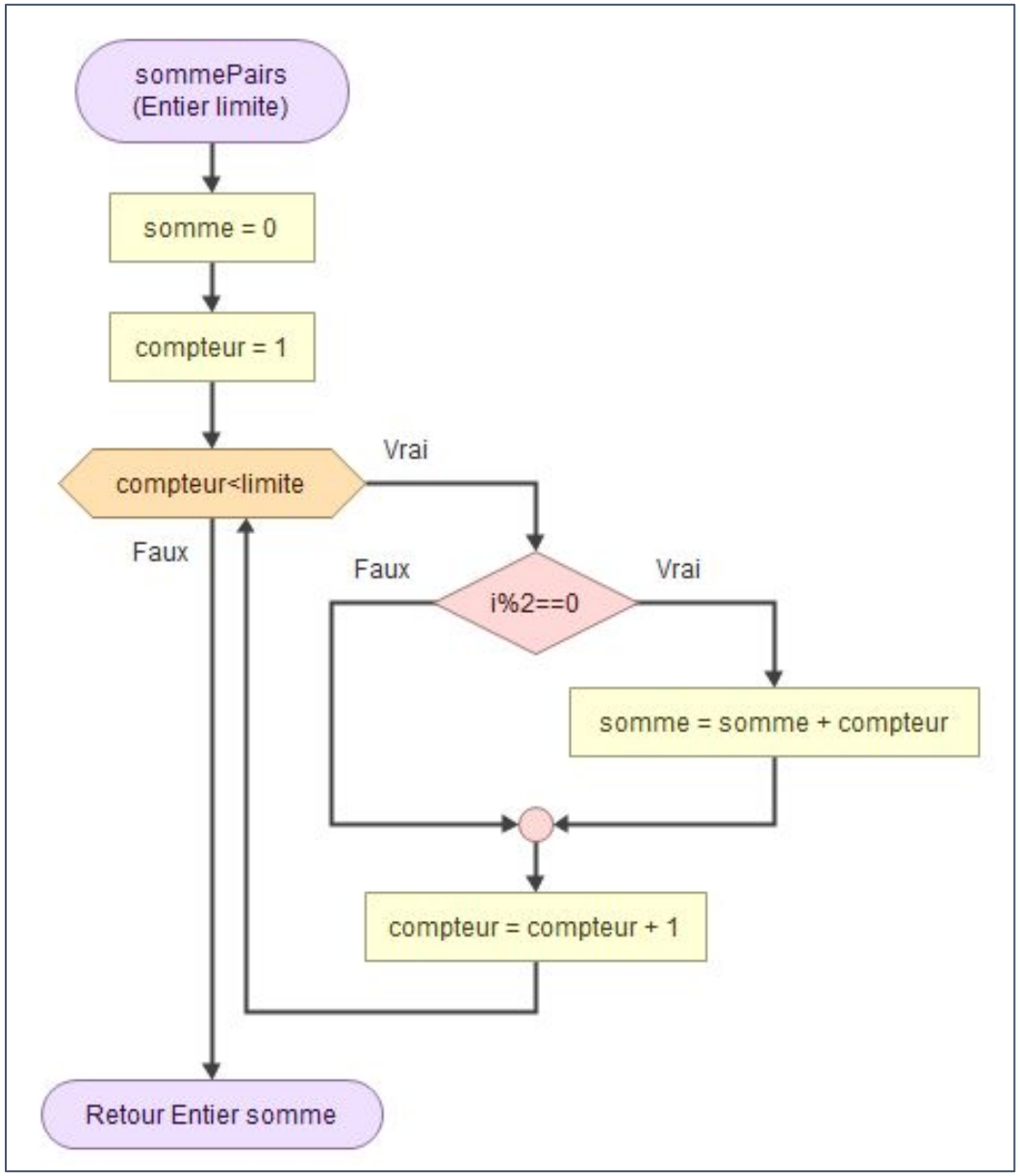
```
def sommePairs(limite) :
    somme = 0
    compteur = 1
    while compteur <= nombre :
        if compteur % 2 == 0 :
            somme += compteur
        compteur += 1
    return somme
```

Pseudo-Code

fonction sommePairs:
Entrées : limite du parcours
Sortie : somme du parcours
initialiser un compteur à 1
tant que le compteur est plus petit que la limite faire
    si le compteur est pair
        ajouter le compteur à la somme
    fin si
    incrémenter le compteur de 1
fin tant que
retourner la somme

Caractéristiques d’un algorithme

Caractéristique	Algorithme
Définition	Un ensemble d'étapes d'une procédure ou d'une méthode de résolution de problèmes.
Syntaxe	Aucune règle de syntaxe spécifique ; décrit en langage clair ou sous forme de diagramme.
Exécution	Pas directement exécutable ; c'est un plan pour résoudre un problème.
Usage	Utilisé pour la planification, la conception et la communication d’approches de résolution de problèmes.



Caractéristiques du pseudocode

Caractéristique	Pseudocode
Définition	Représentation de haut niveau d'un algorithme utilisant un mélange de format technique et de texte lisible par l'homme.
Syntaxe	Utilise des constructions de langage structuré telles que des boucles, des conditions et des fonctions, mais sans règles de syntaxe strictes.
Exécution	Non exécutable seul ; il fournit un aperçu de haut niveau de la logique de la solution.
Usage	Utilisé pour communiquer et documenter un algorithme avant d'écrire du code.

fonction sommePairs:
Entrées : limite du parcours
Sortie : somme du parcours
initialiser un compteur à 1
tant que le compteur est plus petit que la limite
faire
    si le compteur est pair
        ajouter le compteur à la somme
    fin si
    incrémenter le compteur de 1
fin tant que
retourner la somme

Caractéristiques d’un programme informatique

Caractéristique	Programme
Définition	Un ensemble d'instructions écrites pour les ordinateurs dans un **langage de programmation** particulier pour implémenter un algorithme ou un pseudocode
Syntaxe	Doit respecter la syntaxe et les règles précises du langage de programmation choisi.
Exécution	Exécutable par un ordinateur ; il peut être compilé ou interprété pour produire un programme fonctionnel.
Usage	Utilisé pour l’exécution réelle et la résolution du problème en question.

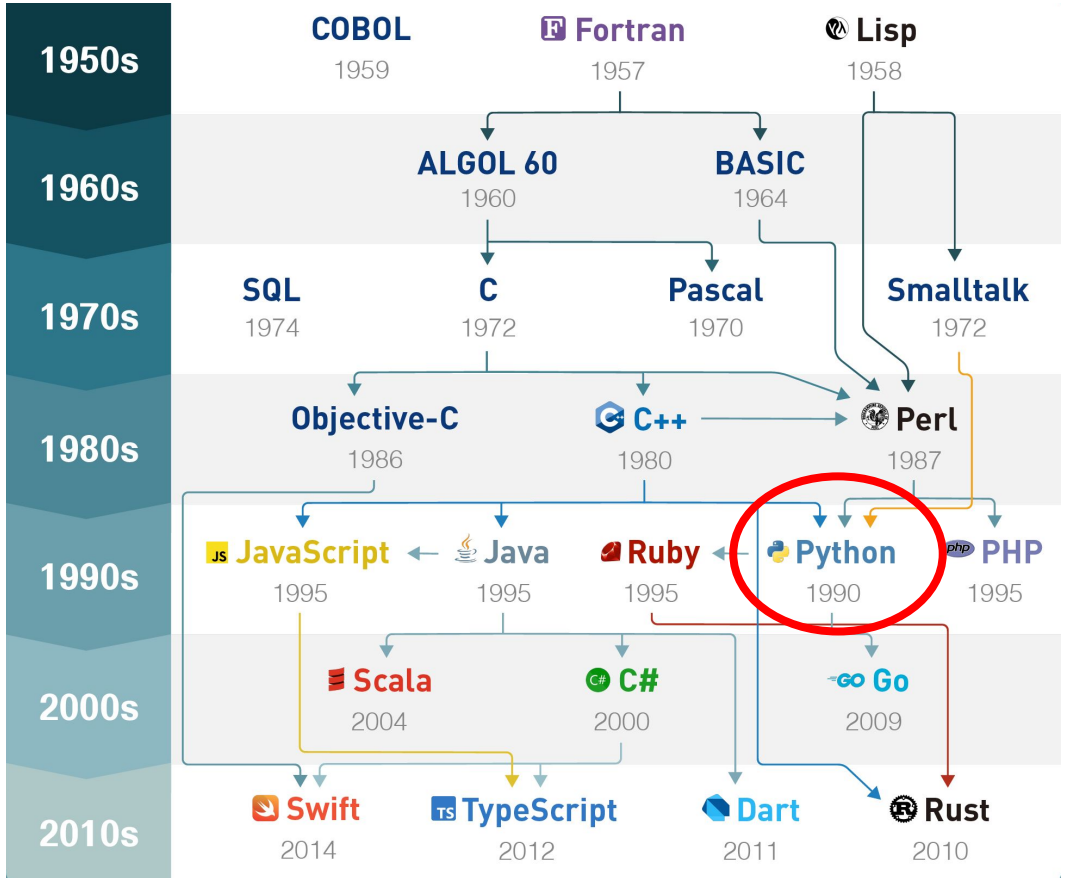
```
def sommePairs(limite) :
  somme = 0
  compteur = 1
  while compteur <= nombre :
    if compteur % 2 == 0 :
      somme += compteur
    compteur += 1
  return somme
```

Tableau récapitulatif

Caractéristique	Algorithme	Pseudocode	Programme
Définition	Un ensemble d'étapes d'une procédure ou d'une méthode de résolution de problèmes.	Représentation de haut niveau d'un algorithme utilisant un mélange de format technique et de texte lisible par l'homme.	Un ensemble d'instructions écrites pour les ordinateurs dans un langage de programmation particulier pour implémenter un algorithme ou un pseudocode
Syntaxe	Aucune règle de syntaxe spécifique ; décrit en langage clair ou sous forme de diagramme.	Utilise des constructions de langage structuré telles que des boucles, des conditions et des fonctions, mais sans règles de syntaxe strictes.	Doit respecter la syntaxe et les règles précises du langage de programmation choisi.
Exécution	Pas directement exécutable ; c'est un plan pour résoudre un problème.	Non exécutable seul ; il fournit un aperçu de haut niveau de la logique de la solution.	Exécutable par un ordinateur ; il peut être compilé ou interprété pour produire un programme fonctionnel.
Usage	Utilisé pour la planification, la conception et la communication d'approches de résolution de problèmes.	Utilisé pour communiquer et documenter un algorithme avant d'écrire du code.	Utilisé pour l'exécution réelle et la résolution du problème en question.

De nombreux langages de programmation existent

- Les langages partagent des caractéristiques (historiques) communes.
- Langage ancien ≠ langage désuet
- Le but d'un langage de programmation est de permettre au programmeur de (facilement) mettre en œuvre sa pensée.
- Or, différentes manières de penser ⇒ différents langages créés !



Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Partie I

Les bases de la programmation impérative et procédurale – avec Python

1. Préambule
2. Structures de données simples
3. Structures de contrôle et itératives
4. Fonctions

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

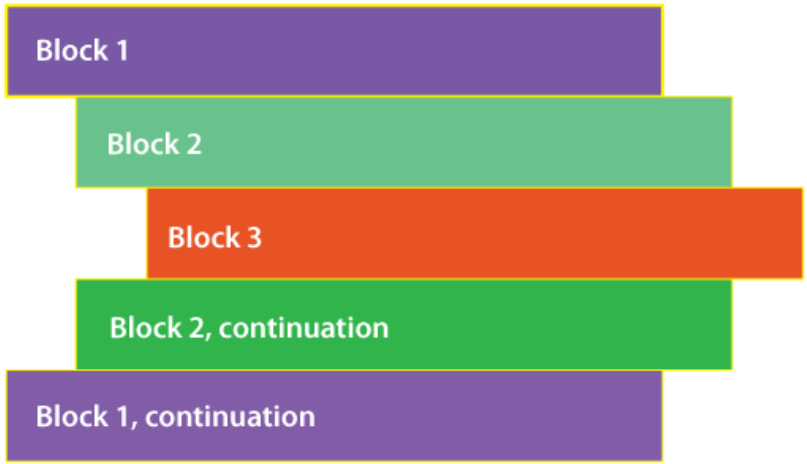
4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

- Programmation :
  - expression d'un algorithme (résolvant un problème) dans un langage informatique donné, *e.g.* `Python`.
  - On parle de *développeur* ou de *codeur* pour une personne qui écrit un programme informatique.
- Programmation impérative :
  - exprimer le **"comment"**
  - manipulation explicite de la mémoire par des instructions
- Programmation procédurale :
  - utilisation de **fonctions** (on parle aussi de *procédures*, *méthodes*)
  - permettant la réutilisation d'instructions
- Programme = exécution d'une **séquence d'instructions**
  - séquence : ordre à respecter
    - instructions les unes à la suite des autres
  - instruction : passer d'un état initial à un état final
    - état $\Leftrightarrow$ celui d'un ensemble de **variables** situées en mémoire.
  - Fin du programme lorsque plus d'instruction à exécuter.

Instruction et bloc d'instructions

- Contrairement à Java ou à C, en Python il n'y a pas :
  - de point-virgule `;` à la fin d'une instruction
  - d'accolades `{ }` entre chaque bloc

- En `Python`, on construit un bloc avec le symbole "deux points" `:` et en **indentant** :
  - **indenter**, c'est mettre une tabulation avant une instruction
  - une **tabulation** peut être ajoutée avec la touche tab ou ⇤⇥



Les commentaires

- Les commentaires sont ignorés par le compilateur. Ils servent notamment à :
  - avoir un rôle informatif pour le développeur (*e.g.* pour expliciter un point précis).
  - retirer temporairement un bout de code de la compilation et de l'exécution (attention à la propreté du code !)

- Pour écrire un commentaire, on utilise :
  - le symbole # : tout ce qui se trouve après sur la même ligne est considéré comme commentaire.
  - on peut commenter plusieurs lignes à la fois : à l'aide de la souris, sélectionner plusieurs ligne, puis appuyer sur ctrl + /

Les entrées/sorties (input/output)

Entrées

- par exemple le clavier
- On parle de **lecture** du clavier par l'ordinateur.
- ↯ L'utilisateur quant à lui *écrit* au clavier.
- fonction pour demander une entrée à l'utilisateur : `texte = input()`

Sorties

- par exemple l'écran
- On parle d'**écriture** sur l'écran par l'ordinateur.
- ↯ L'utilisateur quant à lui *lit* à l'écran.
- fonction pour afficher en `Python` :
  - `print("texte")`
  - pour un retour à la ligne, on ajoute le caractère `"\n"` au texte

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

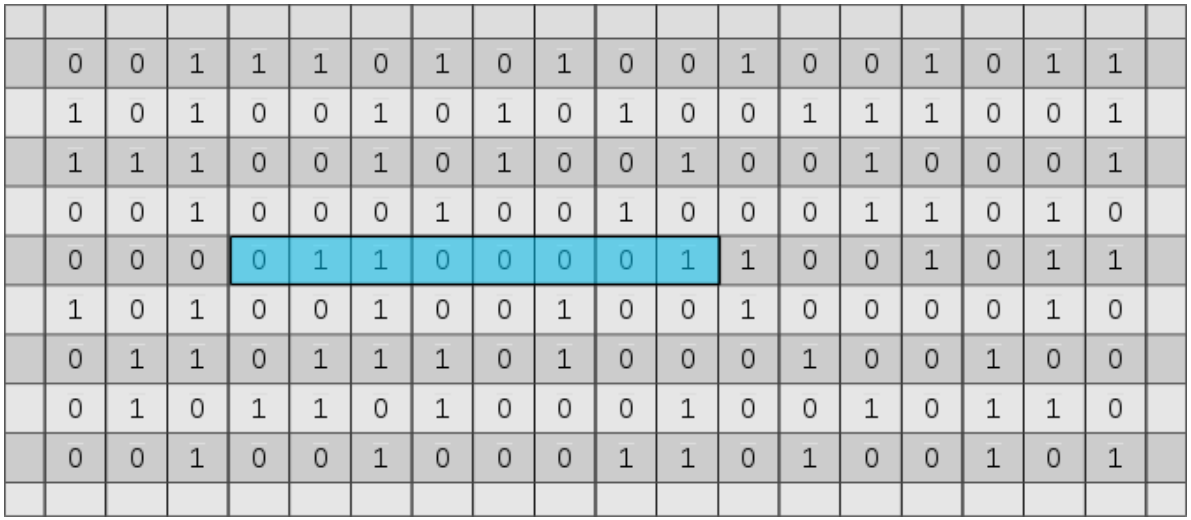
3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Définition (1/2)

Une variable est un **espace mémoire** dans lequel une **valeur** est stockée.

- la **mémoire** c'est un ensemble de bits
  - bit = une cellule + sa valeur binaire (0 ou 1)
  - *e.g.* 16 Go de mémoire $\Leftrightarrow 16 \, x \, 2^{30} \, x \, 8$ bits ($\approx$ 138 milliards)



- ici, en bleu, l'espace mémoire de 1 octet (8 bits) et sa valeur :

0	1	1	0	0	0	0	1

Définition (2/2)

Une variable est un **espace mémoire** dans lequel une **valeur** est stockée.
Une variable possède un **type** et un **nom**.

Définition (2/2)

Une variable est un **espace mémoire** dans lequel une **valeur** est stockée.
Une variable possède un **type** et un **nom**.

Le **type**

- détermine la place mémoire occupée par la variable
- et donc... les valeurs possibles/acceptables qui pourront être stockées (une à la fois !) dans la variable.

Définition (2/2)

Une variable est un **espace mémoire** dans lequel une **valeur** est stockée.
Une variable possède un **type** et un **nom**.

Le **type**

- détermine la place mémoire occupée par la variable
- et donc... les valeurs possibles/acceptables qui pourront être stockées (une à la fois !) dans la variable.

- permet d'interpréter la valeur

  e.g. `0|1|1|0|0|0|0|1`
  - si type = `int` : correspond à l'entier 97
  - si type = `char` : correspond au caractère 'a'
  - *etc.*

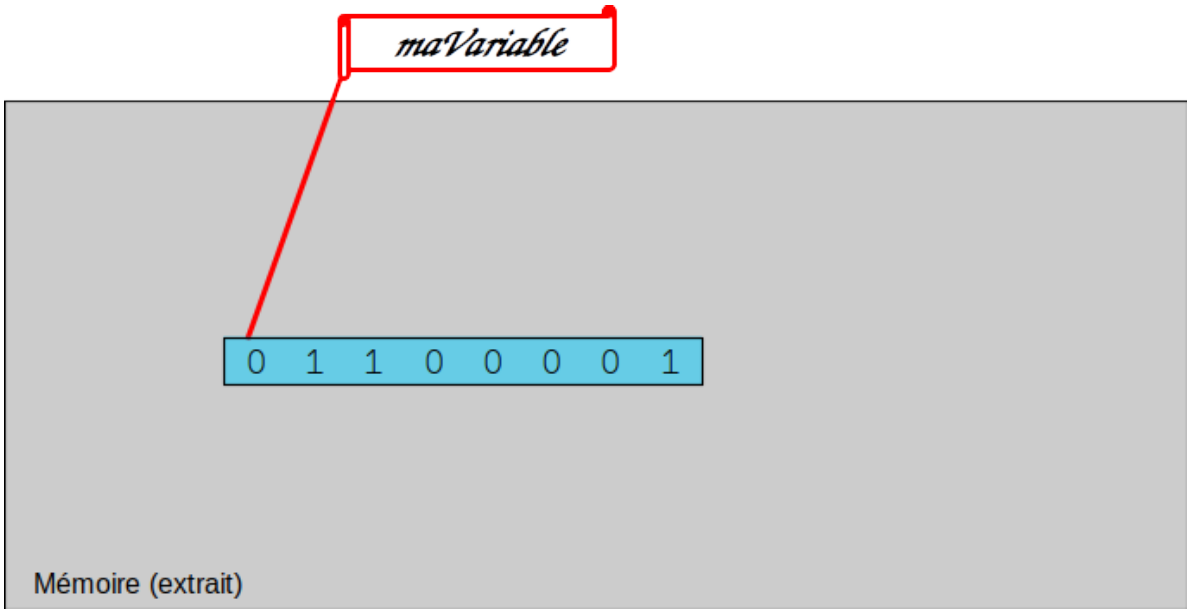
Définition (2/2)

Une variable est un **espace mémoire** dans lequel une **valeur** est stockée.
Une variable possède un **type** et un **nom**.

Le **nom**

- permet de faire référence de manière symbolique à cet espace mémoire dans le code.

$\Leftrightarrow$ identificateur unique.



Manipulations de variables

Le langage `Python` utilise le **typage dynamique** :

- il n'est pas nécessaire de **declarer** une variable avec son type avant de l'affecter
- la gestion du type d'une variable est laissée à l'ordinateur qui réalise cette opération **"à la volée"**
- le **typage dynamique** s'oppose au **typage statique** (Java, C)
- le type d'une variable n'est **pas figé**, on peut remplacer sa valeur par une autre d'un type différent

Syntaxe (Accès en écriture $\Leftrightarrow$ **affectation**)

nomDeLaVariable $=$ *valeur*

↯ la $\mathbf{1^{ère}}$ **affectation** s'appelle l'**initialisation**

Syntaxe (Accès en lecture)

nomDeLaVariable

Manipulations de variables

Exemples :

Code

```
1 variable1 = 12.3 # initialisation (1ère affectation)
2 variable2 = 4.0 # initialisation
3 variable2 = variable1 / 3 + 5 # affectation
4 variable1 = "bonjour" # affectation d'une valeur avec type différent
```

Le typage dynamique c'est **pratique** :

- l'écriture du code est plus **simple** et plus **rapide**
- il n'est pas nécessaire d'effectuer systématiquement des ***'cast'*** (changement de type à la volée)

Mais cela a aussi des implications **néfastes** :

- une **surconsommation de mémoire** correspondant à l'encodage 'à la volée' du type dans la valeur
- une **performance d'exécution** de programme globalement en deçà d'autres langages de plus bas niveau à typage statique (Java, C)

Constantes

- En `Python`, il n'y a **pas de véritables constantes** car les variables peuvent être réassignées.
- Cependant, il est de convention d'utiliser des **majuscules** pour indiquer qu'une variable est constante et qu'elle ne doit pas être modifiée

Syntaxe

UNE_CONSTANTE = valeur

Code

```
1 PI = 3.141592654 #constante
2 diametre = 2 * PI * 6371.137
```

Convention de nommage

- caractères valides pour le nom d'un élément :
  - `abcdefghijklmnopqrstuvwxyz`
    `ABCDEFGHIJKLMNOPQRSTUVWXYZ`
    `_ 0123456789`
- un nom ne peut pas commencer par un chiffre

- une constante est tout en Majuscules
- une *classe* commence par une Majuscule
- tout le reste commence par une minuscule : variable, fonction, *etc.*

- un nom peut être composé de plusieurs "mot", dans ce cas :
  - tous les mots sont attachés
  - classes : chaque mot suivant commence par une Majuscule *(CamelCase)*
  - variables et fonctions : chaque mot est séparé par _ *(snake_case)*

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Les types simples en Python

types	description	valeurs
boolean	booléen	`True` ou `False`
int	nombre entier	ex : -23, 0, 378
float	nombre flottant/décimal	ex : -1.287, 0.0, 589.3254
str	string, chaine de caractère	ex : "bonjour le monde", "a", "589"

Conversion de types

- **Conversions explicites**
  - Le *transtypage*, appelé **cast**, est requis lorsqu'il peut y avoir perte d'information ou une ambiguïté.
  - ↯ Terrain glissant → d'où l'obligation d'expliciter !

Syntaxe (fonction de *cast* pour forcer le changement de type)

nouveauType(*nomDeLaVariable*)

Code

```
1 r = 65.4
2 e = int(r)  // e vaut 65  (valeur entière conservée)
3 c = chr(e)  // c vaut 'A'
```

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Plusieurs catégories d'opérateurs

- **opérateurs arithmétiques**
  - +, -, *, /, % (modulo), // (division entière), ** (puissance)
- **opérateurs d'affectation (assignation)**
  - =,
  - +=, -=, *=, /=, %=
- **opérateurs de comparaison**
  - ==, !=, <, <=, >, >=
- **opérateurs logiques**
  - `and`, `or`, `not`, *etc.*

  e.g. `(age > 18) and (choix == 'b')`
- **autres operateurs :**
  - **ensemblistes :** `in` et `not in`
  - **comparaison d'objets :** `is` et `is not`

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Deux structures conditionnelles :

- "Si ..." :
  - cas 1 : " Si ..., alors ... " (`if`)
  - cas 2 : " Si ..., alors ..., Sinon ... " (`if  else`)
  - cas 3 : " Si ..., Sinon si ..., Sinon ..." (`if, elif, else`)

- "Au cas où ..." (`match`)

✎ ces instructions peuvent être imbriquées

if / else

- Si uniquement

Syntaxe

```
if test :
    instruction_cas_test_vrai
```

- Si – Sinon

Syntaxe

```
if test :
    instruction_cas_test_vrai
else :
    instruction_cas_test_faux
```

if / else

Code

```
...

if (age >= 7) and (age <= 77) :
    print("Ce jeu est fait pour vous.")

else :
    print("Essayez un autre jeu...")
...
```

Rappel :

- le ET logique s'écrit **and**
- le OU logique s'écrit **or**
- le NON logique s'écrit **not**

if / else imbriqués

- **Bien indenter le code, sinon ça ne marche pas !**

✎ Penser à la touche tab : 

Code

```
...
if x > 0 :
    # cas avec x strictement positif

else :
    if x < 0  :
        # cas avec x strictement negatif

    else :
        # cas avec x valant 0

...# suite en dehors du if / else principal
```

`if / else` imbriqués

- Éviter d'écrire `elif`... qui prête à confusion ! !

Code

```
1  ...
2  if x > 0 :
3      #cas avec x strictement positif
4
5  elif x < 0 :
6      #cas avec x strictement negatif
7
8  else :
9      #cas avec x valant 0
10 ...
```

Exemple – le type booléen : tout un art !

Supposons la variable age existante

Code (Affectation)

```
1 estAdulte = None
2 if age >= 18 :
3     estAdulte = True
4 else :
5     estAdulte = False
```

Exemple – le type booléen : tout un art !

Supposons la variable age existante

Code (Affectation)

```
1 estAdulte = None
2 if age >= 18 :
3     estAdulte = True
4 else :
5     estAdulte = False
```

→ ou bien :

Code (... pas besoin de else)

```
1 estAdulte = False
2 if age >= 18 :
3     estAdulte = True
```

Exemple – le type booléen : tout un art !

Supposons la variable age existante

Code (Affectation)

```
1 estAdulte = None
2 if age >= 18 :
3     estAdulte = True
4 else :
5     estAdulte = False
```

→ ou bien :

Code (... pas besoin de else)

```
1 estAdulte = False
2 if age >= 18 :
3     estAdulte = True
```

→ ou encore :

Code (... encore plus simple !)

```
1 estAdulte = (age >= 18)
```

Exemple – le type booléen : tout un art !

Supposons la variable age existante

Code (Affectation)

```
1 estAdulte = None
2 if age >= 18 :
3     estAdulte = True
4 else :
5     estAdulte = False
```

→ ou bien :

Code (... pas besoin de else)

```
1 estAdulte = False
2 if age >= 18 :
3     estAdulte = True
```

→ ou encore :

Code (... encore plus simple !)

```
1 estAdulte = (age >= 18)
```

Code (Utilisation)

```
1 if estAdulte : # pas besoin d'écrire if(estAdulte == true)
2     # ...
```

`match`

- En cas de **tests multiples d'égalité**, l'instruction `match`[1] permet l'énumération de différents cas possibles. Cela évite une série de `if` imbriqués.

1. Au début de la structure, l'expression est évaluée et son résultat est comparé (==) aux différents cas (`case`) de valeurs possibles.
2. La clause *optionnelle* `default` ou _ est exécutée lorsque la valeur de l'expression n'est égale à aucun des cas.

1. fonctionne uniquement à partir de la version 3.10 de python

Le match, bien plus clair et simple

Code

```
1  def match(jour):
2    if jour == 1:
3      return 'Lundi'
4    elif jour == 2:
5      return 'Mardi'
6    elif jour == 3:
7      return 'Mercredi'
8    elif jour == 4:
9      return 'Jeudi'
10   elif jour == 5:
11     return 'Vendredi'
12   elif jour == 6:
13     return 'Samedi'
14   elif jour == 7:
15     return 'Dimanche'
16   else:
17     return 'Pas un jour de
           la semaine'
```

Code

```
1  match jour:
2    case 1:
3      return 'Lundi'
4    case 2:
5      return 'Mardi'
6    case 3:
7      return 'Mercredi'
8    case 4:
9      return 'Jeudi'
10   case 5:
11     return 'Vendredi'
12   case 6:
13     return 'Samedi'
14   case 7:
15     return 'Dimanche'
16   case _:
17     return 'Pas un jour de
           la semaine'
```

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

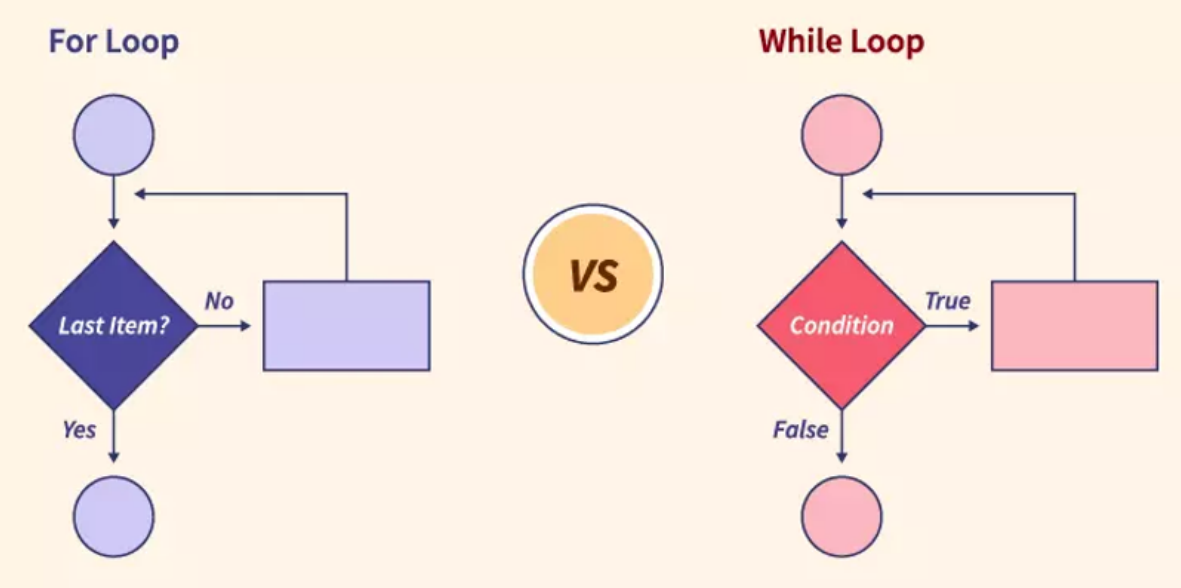
3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Deux structures itératives :

- " Tant que... " (`while`)
- " Pour... " (`for`)

- Ces structures permettent une **exécution répétitive du code**, on dit **en boucle**.
- L'exécution cesse quand la condition de test n'est plus remplie.



while

- On veut répéter une ou plusieurs instructions un nombre **indéterminé** de fois.

Code

```
1 e = 0
2
3 while( e < 5 ) :
4   print("e : ", e)
5   e+=1
```

- tant que l'évaluation du **test** est `True` alors corps de la boucle exécuté

✎ la condition est donc réévaluée après chaque tour de boucle.

while

- On veut répéter une ou plusieurs instructions un nombre **indéterminé** de fois.

Code

```
1 e = 0
2
3 while( e < 5 ) :
4   print("e : ", e)
5   e+=1
```

- tant que l'évaluation du **test** est `True` alors corps de la boucle exécuté

✎ la condition est donc réévaluée après chaque tour de boucle.

Exemple (Affichage)

```
e : 0
e : 1
e : 2
e : 3
e : 4
```

for

- On veut répéter un nombre de fois prédéfini une ou plusieurs instructions.
- On peut utiliser `range(A, B, C)` : une fonction génératrice, elle produit un tableau d'entier dans [A, B[avec un pas de C.
- A (le début) et C (le pas) sont optionnels, par défaut A=0 et C=1. B est obligatoire et défini la limite du parcours.

Code

```
1 #                A   B   C
2 for e in range(0, 5, 2) :
3     print("e : ", e)
```

for

- On veut répéter un nombre de fois prédéfini une ou plusieurs instructions.
- On peut utiliser `range(A, B, C)` : une fonction génératrice, elle produit un tableau d'entier dans [A, B[avec un pas de C.
- A (le début) et C (le pas) sont optionnels, par défaut A=0 et C=1. B est obligatoire et défini la limite du parcours.

Code

```
1 #                A   B   C
2 for e in range(0, 5, 2) :
3     print("e : ", e)
```

Exemple (Affichage)

```
e : 0
e : 2
e : 4
```

`for`

- Une boucle `for` necessite un **itérable** (que nous verrons ensemble plus en détail dans le prochain cours).

- Un itérable est un objet dont on peut parcourir les éléments un par un et effectuer une action sur chacun d'eux. ex :
  - Les objets `range()`
  - Les chaînes de caractères
  - Les listes
  - Les tuples
  - ...

`for` : exemple (chaine de caractères)

Code

```
1 for e in "bonjour" :
2     print(e)
```

Exemple (Affichage)

```
b
o
n
j
o
u
r
```

`for` : exemple (liste d'entiers)

Code

```
1 for e in [3,12,7,9,5,0] :
2     print(e)
```

Exemple (Affichage)

```
3
12
7
9
5
0
```

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Définition

Syntaxe

```
def nomFonction(param1, param2, ...) :
    ...
    # corps de la fonction
    ...
    return valeur
```

- la **première ligne** s'appelle la **signature de la fonction**
- les paramètres (`param1, param2...`) sont dit *"en **entrée** de la fonction"*, ce sont les **input**,
- ...ils sont utilisés dans le ***"corps de la fonction"***,
- ...afin de calculer ***la ou les sorties/retours de la fonction***, ce sont les **output**,
- ...la sortie est **retourné** à l'aide du mot clé `return`

Utilisation

- Découpage d'un programme en sous-programmes

$\Rightarrow$ permet de **diviser un traitement** en sous-traitements.

$\Rightarrow$ permet de **réutiliser du code**.

Utilisation

- Découpage d'un programme en sous-programmes

$\Rightarrow$ permet de **diviser un traitement** en sous-traitements.

$\Rightarrow$ permet de **réutiliser du code**.

- D'une manière générale pour un sous-programme,
  - on parle de **fonction** lorsqu'un résultat est **retourné** (vision Maths),
  - sinon on parle de **procédure**.

Utilisation

Fonction = Boîte à outils

- agissant comme une boîte noire pour l'utilisateur
- qui prend en entrée 0 ou plusieurs paramètres [2]
  - **paramètre** $\Leftrightarrow$ **variable** (type + nom + valeur)
  - ↯ variable locale à la fonction
- qui renvoie généralement un résultat
  - **résultat** $\Leftrightarrow$ **valeur** d'un certain type
- le plus autonome et générique possible
  - utilisable plusieurs fois, réutilisable...
  - prévoir les fois ou ça ne fonctionne pas
  - un comportement parfois différent en fonction des paramètres

$\Rightarrow$ Lors d'un développement, beaucoup de fonctions sont utilisées et créées.

2. On parle aussi d'*arguments*

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

Exemple #1

Code (Définition)

```
1 def somme(a, b) :
2     resultat = a + b
3     return resultat
4 # les variables locales a, b et resultat détruites en fin de bloc
```

Contrairement à Java et C, en Python :

- la signature de la fonction ne donne aucune indication sur le type du retour et des arguments
- il est donc important de donner un nom clair et sans ambiguïté à la fonction, voir d'ajouter une description via un commentaire

Code (Utilisation)

```
1 addition = somme(2, 3)
2 print(addition)
```

Affichage

5

Exemple #2

Code (Définition)

```
1 def carre(nombre) :
2   resultat = nombre * nombre
3   return resultat
```

Code (Utilisations)

```
1 deux   = 2
2 quatre = carre( deux )
3 print( quatre )
4
5 print( carre(3) )
6 print( carre( carre(quatre) ) )
```

Exemple (Affichage)

```
4
9
256
```

1 Préambule

2 Structures de données simples
- Les variables
- Les types simples (primitifs)
- Les opérateurs

3 Structures de contrôle et itératives
- Les structures de contrôle
- Les structures itératives

4 Fonctions
- Définition et utilisation d'une fonction
- Quelques exemples
- La portée des variables

- En `Python`, nous pouvons déclarer des variables n'importe où dans notre script : au début du script, à l'intérieur de boucles, au sein de nos fonctions, etc.

- L'endroit où on définit une variable dans le script va déterminer l'endroit à partir duquel la variable va être accessible, c'est-à-dire, utilisable.

- Ainsi, le terme de "portée des variables" sert à désigner les différents espaces dans le script dans lesquels une variable est accessible c'est-à-dire utilisable. En Python, une variable peut avoir une portée globale ou une portée locale.

Les variables **définies dans une fonction** sont appelées **variables locales :**

- Elles ne peuvent être utilisées que localement c'est-à-dire qu'à l'intérieur de la fonction qui les a définies.
- Tenter d'appeler une variable locale depuis l'extérieur de la fonction qui l'a définie provoquera une erreur.

Code

```
1 def portee() :
2     variable = 5
3     print(variable) #accessible car dans la même fonction
4
5 portee() #appel de la fonction
6 print(variable) #ERREUR, n'est plus accessible
```

Exemple (Affichage)

5
NameError : name 'variable' is not defined

F.A.Q

"Monsieur, coder, ça n'est pas facile !"

- Le code ne viendra pas tout seul et ne viendra pas tout de suite, encore moins en s'énervant ou en attendant que ça se passe

  Pas d'inquiétude ! ! !

  → Commencez par écrire **sur papier** le pseudo-code / vos idées
  → Puis dans un second temps, transformez le en code, pas à pas

"Monsieur, pourquoi perdre son temps à apprendre l'informatique alors qu'avec un LLM, ça marche vachement bien ?"

- Aujourd'hui, les **développeurs** utilisent systématiquement les prompt LLM pour écrire du code.
- Cependant, ils **comprennent** le code proposé par le LLM et sont capables de le déboguer et de le raffiner
- Pour info, on pensait que le métier de développeur aller disparaître, finalement, il a juste évolué